# 3.4 Manipulating Equations

## 3.4.1 The Representation of Equations and Solutions

*Mathematica* treats equations as logical statements. If you type in an equation like `x^2 + 3x == 2`, *Mathematica* interprets this as a logical statement which asserts that `x^2 + 3x` is equal to `2`. If you have assigned an explicit value to `x`, say `x = 4`, then *Mathematica* can explicitly determine that the logical statement `x^2 + 3x == 2` is `False`.

If you have not assigned any explicit value to `x`, however, *Mathematica* cannot work out whether `x^2 + 3x == 2` is `True` or `False`. As a result, it leaves the equation in the symbolic form `x^2 + 3x == 2`.

You can manipulate symbolic equations in *Mathematica* in many ways. One common goal is to rearrange the equations so as to "solve" for a particular set of variables.

Here is a symbolic equation.

```
In[1]:= x^2 + 3x == 2

                 2
Out[1]= 3 x + x  == 2
```

You can use the function `Roots` to rearrange the equation so as to give "solutions" for `x`. The result, like the original equation, can be viewed as a logical statement.

```
In[2]:= Roots[%, x]

              -3 + Sqrt[17]           -3 - Sqrt[17]
Out[2]= x == ------------- || x == -------------
                    2                       2
```

The quadratic equation `x^2 + 3x == 2` can be thought of as an implicit statement about the value of `x`. As shown in the example above, you can use the function `Roots` to get a more explicit statement about the value of `x`. The expression produced by `Roots` has the form `x ==` $r_1$ `||` `x ==` $r_2$. This expression is again a logical statement, which asserts that either `x` is equal to $r_1$, or `x` is equal to $r_2$. The values of `x` that are consistent with this statement are exactly the same as the ones that are consistent with the original quadratic equation. For many purposes, however, the form that `Roots` gives is much more useful than the original equation.

You can combine and manipulate equations just like other logical statements. You can use logical connectives such as `||` and `&&` to specify alternative or simultaneous conditions. You can use functions like `LogicalExpand` to simplify collections of equations.

For many purposes, you will find it convenient to manipulate equations simply as logical statements. Sometimes, however, you will actually want to use explicit solutions to equations in other calculations. In such cases, it is convenient to convert

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

equations that are stated in the form *lhs* `==` *rhs* into transformation rules of the form *lhs* `->` *rhs*. Once you have the solutions to an equation in the form of explicit transformation rules, you can substitute the solutions into expressions by using the `/.` operator.

`Roots` produces a logical statement about the values of `x` corresponding to the roots of the quadratic equation.

```
In[3]:= Roots[ x^2 + 3x == 2, x ]

               -3 + Sqrt[17]           -3 - Sqrt[17]
Out[3]= x == ------------- || x == -------------
                     2                       2
```

`ToRules` converts the logical statement into an explicit list of transformation rules.

```
In[4]:= {ToRules[ % ]}

                  -3 + Sqrt[17]          -3 - Sqrt[17]
Out[4]= {{x -> -------------}, {x -> -------------}}
                        2                      2
```

You can now use the transformation rules to substitute the solutions for `x` into expressions involving `x`.

```
In[5]:= x^2 + a x /. %

                                                  2
           a (-3 + Sqrt[17])   (-3 + Sqrt[17])
Out[5]= {----------------- + ----------------,
                   2                   4

                                        2
         a (-3 - Sqrt[17])   (-3 - Sqrt[17])
         ----------------- + ----------------}
                 2                   4
```

The function `Solve` produces transformation rules for solutions directly.

```
In[6]:= Solve[ x^2 + 3x == 2, x ]

                  -3 + Sqrt[17]          -3 - Sqrt[17]
Out[6]= {{x -> -------------}, {x -> -------------}}
                        2                      2
```

When `Solve` cannot find explicit solutions, it leaves the result in a symbolic form in terms of the functions `Roots` and `ToRules`.

```
In[7]:= Solve[ x^5 + 5x + 1 == 0, x ]

                                      5
Out[7]= {ToRules[Roots[5 x + x  == -1, x]]}
```

If you apply `N`, `Roots` finds numerical roots, and `ToRules` converts the result into transformation rules.

```
In[8]:= N[ % ]
Out[8]= {{x -> -1.0045 - 1.06095 I},
     {x -> -1.0045 + 1.06095 I}, {x -> -0.199936},
     {x -> 1.10447 - 1.05983 I},
     {x -> 1.10447 + 1.05983 I}}
```